

Next-Generation Foundation Model Architectures

Advanced LLM Research and Next-Generation NLP: Day 1

Naeemullah Khan

naeemullah.khan@kaust.edu.sa



جامعة الملك عبد الله
للعلوم والتقنية

King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

October 5, 2026

1. Motivation
2. Learning Outcomes
3. Beyond Attention: Recurrent Mixers
 1. Linear Attention
 2. State-Space Models
 3. Other Families: RWKV, xLSTM, Hyena
4. Hybrid Architectures
5. Mixture-of-Experts at Frontier Scale
6. Efficient Attention
 1. Faster Kernels: FlashAttention-3 and 4
 2. Sliding-Window Attention
 3. Learned Sparse Attention

- 4. Ring Attention and Context Parallelism
- 7. Long-Context Architectures
 - 1. Extending Positions
 - 2. Attention Sinks
 - 3. Memory-Augmented Transformers
 - 4. Training and Evaluating at 1M Tokens
- 8. Scaling Laws and Their Limits
- 9. Emerging Multimodal Architectures
- 10. Summary

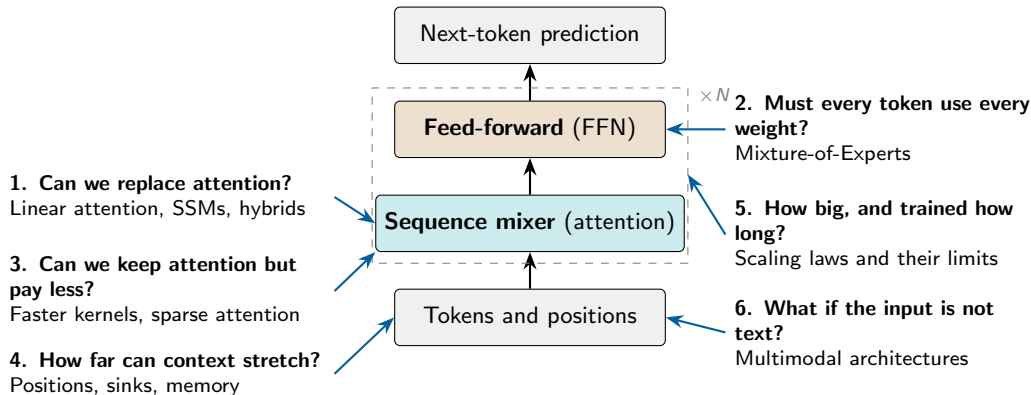
The Transformer Was Not Replaced. It Was Re-plumbed.

- ▶ The block from 2017 is still recognisable in every frontier model of 2026.
- ▶ What changed is **where the cost goes**. Three pressures drive almost every design in this lecture:
 - **Memory per token.** Attention keeps every past token in a cache that grows with context.
 - **Compute per token.** Most parameters sit in feed-forward layers that every token pays for.
 - **Context length.** One million tokens is now a product feature, not a research demo.
- ▶ The answers are **hybrids**, **sparsity** and **compression**, not a new architecture.

Frontier MoE models went from about 28% of parameters active per token (Mixtral, 2024) to about 3 to 5% in 2026.

Raschka, "The Big LLM Architecture Comparison" (2025, updated 2026). Jiang et al., "Mixtral of Experts," [arXiv:2401.04088](https://arxiv.org/abs/2401.04088). Kimi K3 and DeepSeek-V4 model configurations (2026).

Today's Map: One Block, Six Questions



By the end of this session, you should be able to:

- ▶ **Derive** linear attention as a recurrence, and explain how gating and the delta rule turn it into the mixers used in 2026 hybrids.
- ▶ **Compare** state-space models, RWKV, xLSTM and Hyena against a Transformer baseline on recall, throughput and scale.
- ▶ **Analyse** Mixture-of-Experts routing, load balancing and serving cost at frontier scale.
- ▶ **Explain** how FlashAttention-3 and 4, sliding windows and learned sparse attention cut the cost of attention.
- ▶ **Evaluate** long-context methods and tell a claimed context window from an effective one.
- ▶ **Apply** scaling laws to a training budget, and state where they stop holding.
- ▶ **Describe** the current design choices in natively multimodal models.

Next-Generation Architectures: **Beyond Attention**

Does the sequence mixer have to be attention at all?

- Softmax attention is paid for twice: once in **training compute**, once in **inference memory**.

	Training	State at inference
Classic RNN (LSTM)	sequential	fixed, $O(1)$
Softmax attention	parallel, $O(L^2)$	KV cache, $O(L)$
What we want	parallel, $O(L)$	fixed, $O(1)$

- Albert Gu's picture: a Transformer is a *database* that stores every token. A recurrent model is a *brain* that keeps a compressed state.
- The database gives exact recall. The brain gives constant cost. **This section is about getting both.**

Table after Nayebe and Gormley, CMU 10-423 Lecture 21 (2026). Gu, "On the Tradeoffs of State Space Models and Transformers" (2025), <https://goombalab.github.io/blog/2025/tradeoffs/>.

- ▶ Replace the softmax similarity with a kernel, $\text{sim}(q, k) = \phi(q)^\top \phi(k)$. The sum over keys then factors out of the query:

$$o_i = \frac{\sum_{j \leq i} \phi(q_i)^\top \phi(k_j) v_j}{\sum_{j \leq i} \phi(q_i)^\top \phi(k_j)} = \frac{\phi(q_i)^\top \overbrace{\sum_{j \leq i} \phi(k_j) v_j}^{\text{does not depend on } i\text{'s query}}}{\phi(q_i)^\top \sum_{j \leq i} \phi(k_j)}$$

- ▶ In matrix form this is only associativity: $(QK^\top)V = Q(K^\top V)$.
- ▶ Cost falls from $O(L^2d)$ to $O(Ld^2)$. Stanford's CS336 calls the trick *“very silly, but surprisingly important”*.
- ▶ Modern variants drop the normaliser and the feature map, and normalise the output instead.

Katharopoulos et al., “Transformers are RNNs,” ICML 2020, [arXiv:2006.16236](#). Hashimoto, Stanford CS336 Lecture 4 (2026).
Yang et al., [arXiv:2406.06484](#).

Read the sum causally and it becomes a recurrence:

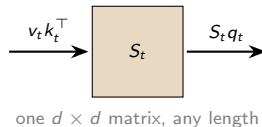
$$S_t = S_{t-1} + v_t k_t^\top, \quad o_t = S_t q_t$$

- ▶ $S_t \in \mathbb{R}^{d \times d}$ is a **fast-weight matrix**. Each token writes one rank-1 update into it.
- ▶ **Train** in the parallel form. **Decode** in the recurrent form, with no cache that grows.
- ▶ Every model in this section changes only two things: what multiplies S_{t-1} , and what gets written.

Softmax attention



Linear attention



Katharopoulos et al., [arXiv:2006.16236](https://arxiv.org/abs/2006.16236). Notation follows Yang et al., "Parallelizing Linear Transformers with the Delta Rule," [arXiv:2406.06484](https://arxiv.org/abs/2406.06484).

- ▶ Ask the state for the value stored under key k_j :

$$S k_j = v_j + \underbrace{\sum_{i \neq j} (k_i^\top k_j) v_i}_{\text{interference}}$$

- ▶ Retrieval is exact only for orthogonal keys, and $\mathbb{R}^d$ holds only d of them. Errors pile up with length.
- ▶ Writes are **additive**. The memory never forgets and never corrects a stale entry.
- ▶ The loss is concentrated: **6.4% of tokens** are recall lookups, and they explain **82%** of the perplexity gap to attention.

Two abilities are missing: **forgetting** and **overwriting**. The next two slides add them one at a time.

Yang, “DeltaNet Explained (Part I)” (2024), <https://sustcsonglin.github.io/blog/2024/deltanet-1/>. Arora et al., “Zoology,” [arXiv:2312.04927](https://arxiv.org/abs/2312.04927).

- ▶ Multiply the old state by a decay before writing:

$$S_t = G_t S_{t-1} + v_t k_t^\top$$

Model	Decay G_t	Depends on the input?
RetNet (2023)	one fixed scalar γ per head	no
Mamba-2 (2024)	scalar γ_t	yes
Gated Linear Attention (2023)	per-channel $\text{Diag}(\alpha_t)$	yes

- ▶ Data-dependent gates close most of the language-modelling gap. At 1.3B parameters GLA reaches perplexity 17.22 against 16.85 for a tuned Transformer.
- ▶ Fixed decay did not survive. In a 2025 study, RetNet gives near-zero recall inside hybrids.
- ▶ **What a gate cannot do:** it scales the whole memory. It cannot fix one wrong association.

Yang et al., “Gated Linear Attention Transformers,” [arXiv:2312.06635](#), Table 2. “A Systematic Analysis of Hybrid Linear Attention,” [arXiv:2507.06457](#).

Fix 2: Learn to Overwrite (the Delta Rule)

- Treat the state as a regression model that should map $k_t \mapsto v_t$, and take **one gradient step per token** on $\mathcal{L}_t(S) = \frac{1}{2} \|S k_t - v_t\|^2$:

$$S_t = S_{t-1} - \beta_t (S_{t-1} k_t - v_t) k_t^\top = S_{t-1} (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$$

- The red factor **erases** whatever was stored along the current key. Then the new value is written.
- Plain linear attention is the same step on a loss that never looks at what is already stored.
- **DeltaNet** solves the hardest synthetic recall setting (MQAR) at 100%.
- Its weakness mirrors Fix 1. With no decay, clearing the whole memory takes d steps.

Schlag et al., “Linear Transformers Are Secretly Fast Weight Programmers,” ICML 2021. Yang et al., “Parallelizing Linear Transformers with the Delta Rule over Sequence Length,” NeurIPS 2024, [arXiv:2406.06484](#).

$$S_t = S_{t-1} (\alpha_t (I - \beta_t k_t k_t^\top)) + \beta_t v_t k_t^\top$$

- **Gate**: fast, global erase.
- **Delta rule**: targeted edit of one key.
- The two are complementary, not alternatives.
- This is the mixer inside **Qwen3-Next** and **Qwen3.5**. Kimi's **KDA** uses a per-channel gate in place of the scalar.

1.3B, 100B tokens	PPL ↓	Recall ↑	Long ↑
Mamba-2	16.56	29.8	13.5
DeltaNet	17.72	26.2	13.6
Gated DeltaNet	16.42	30.6	16.6
+ sliding-window attn.	16.07	40.1	17.8

Recall: average of recall-intensive tasks. Long: LongBench average.

Yang, Kautz, Hatamizadeh, "Gated Delta Networks," ICLR 2025, [arXiv:2412.06464](#). Table as compiled in Yang's DeltaNet talk (2025), slide 27. Kimi Team, [arXiv:2510.26692](#).

- ▶ A recurrence that is cheap on paper can still be slow on a GPU. Step-by-step updates use no matrix multiplies, so the tensor cores sit idle.

Form	Sequential steps	Cost	Used for
Recurrent	L	$O(Ld^2)$	decoding
Fully parallel	1	$O(L^2d)$	short sequences
Chunkwise, chunk C	L/C	$O(LCd + Ld^2)$	training

- ▶ Inside a chunk: ordinary masked attention. Across chunks: pass the state. In practice C is 64 to 256.
- ▶ More FLOPs than the recurrence, yet faster in wall-clock time. **Hardware use beats FLOP count.**
- ▶ The delta rule needs one more trick: its chain of erase matrices is kept in a compact form (the WY representation).

Yang et al., [arXiv:2312.06635](https://arxiv.org/abs/2312.06635) and [arXiv:2406.06484](https://arxiv.org/abs/2406.06484). Library:
<https://github.com/fla-org/flash-linear-attention>.

- ▶ Linear attention started from attention and removed the cache. State-space models (SSMs) started from **control theory** and arrived at the same recurrence.

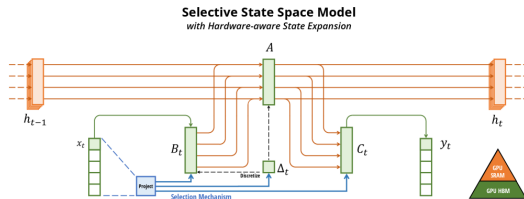
- ▶ Discretise a linear system $h'(t) = Ah(t) + Bx(t)$, $y(t) = Ch(t)$:

$$h_t = \bar{A} h_{t-1} + \bar{B} x_t, \quad y_t = C h_t$$

- ▶ **S4** (2021): fixed A, B, C make the whole map one long convolution, so training is parallel. It solved long-range benchmarks that Transformers failed, such as Path-X at length 16k.
- ▶ The catch is *time-invariance*. One kernel treats every token alike. The model cannot keep or drop a token because of **what it is**.

Gu et al., "Efficiently Modeling Long Sequences with Structured State Spaces," [arXiv:2111.00396](https://arxiv.org/abs/2111.00396).

- **One change:** make B , C and the step size Δ functions of the current token.
- Read Δ_t as a gate. Large: reset the state toward this token. Small: ignore it and hold.
- Selectivity breaks the convolution, so Mamba runs a **hardware-aware parallel scan** instead.
- Payoff reported in the paper: about $5\times$ the generation throughput of a same-size Transformer.



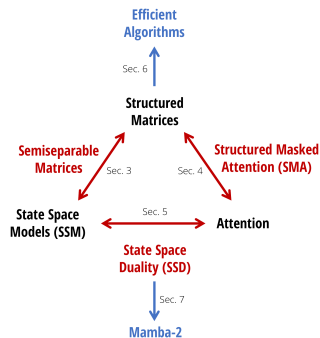
Gu and Dao, "Mamba: Linear-Time Sequence Modeling with Selective State Spaces," [arXiv:2312.00752](https://arxiv.org/abs/2312.00752), Figure 1.

- Write an SSM's whole sequence map as one matrix, $y = Mx$. That matrix is **semiseparable**.
- Apply it as a recurrence: linear time. Apply it as a masked attention matrix: quadratic time. **Same function, two algorithms.**
- In our notation Mamba-2 is simply

$$S_t = \gamma_t S_{t-1} + v_t k_t^\top$$

which is *gated linear attention* with a scalar gate.

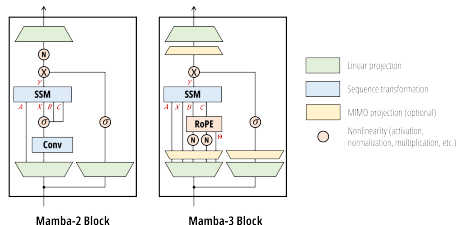
- So “attention versus SSM” is a choice of **mask structure** inside one family.



Dao and Gu, “Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality,” [arXiv:2405.21060](https://arxiv.org/abs/2405.21060), Figure 1.

Mamba-3 (2026): Designed for Inference First

- **Better discretisation.** A trapezoidal rule replaces the separate conv layer.
- **Complex-valued state.** The state can *rotate*, which scalar decay cannot. It acts as a data-dependent RoPE.
- **MIMO.** More compute per decode step, same state size.



At 1.5B	Mamba-3	Mamba-2
Perplexity	10.24	10.47
Parity task	100%	0.9%

Results stop at 1.5B parameters. No shipped production model used Mamba-3 as of October 2026.

Lahoti et al., "Mamba-3: Improved Sequence Modeling using State Space Principles," ICLR 2026, [arXiv:2603.15569](https://arxiv.org/abs/2603.15569). Figure: Goomba Lab blog, "Mamba-3 Part 1" (2026).

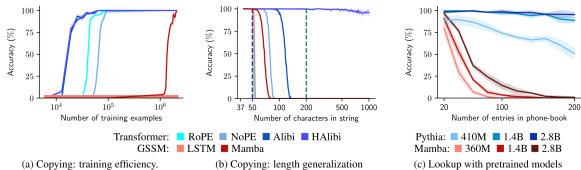
Every mixer so far is a rule for updating a matrix state, read out as $o_t = S_t q_t$.

Model	State update	What it adds
Linear attention	$S_t = S_{t-1} + v_t k_t^\top$	nothing: write only
RetNet	$S_t = \gamma S_{t-1} + v_t k_t^\top$	fixed decay
Mamba-2	$S_t = \gamma_t S_{t-1} + v_t k_t^\top$	input-dependent decay
GLA	$S_t = S_{t-1} \text{Diag}(\alpha_t) + v_t k_t^\top$	per-channel decay
DeltaNet	$S_t = S_{t-1}(I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$	targeted overwrite
Gated DeltaNet	$S_t = S_{t-1} \alpha_t (I - \beta_t k_t k_t^\top) + \beta_t v_t k_t^\top$	decay and overwrite

- Each transition stays **diagonal plus low-rank**. That structure is what keeps chunkwise training possible.

After Yang et al., [arXiv:2406.06484](#), Table 2, and Yang, Kautz, Hatamizadeh, [arXiv:2412.06464](#).

- ▶ A fixed-size state is a **summary**. What the summary drops is gone.
- ▶ **Theory:** any recurrent model needs $\Omega(N)$ bits of state to solve multi-query recall over N pairs.
- ▶ **Practice:** Transformers learn to copy from far fewer examples and generalise to longer strings.
- ▶ Better update rules raise the ceiling. They do not remove it.



Transformers (blue) against fixed-state models (red) on copying and lookup.

Jelassi et al., "Repeat After Me: Transformers are Better than State Space Models at Copying," [arXiv:2402.01032](https://arxiv.org/abs/2402.01032), Figure 1.
Arora et al., "Simple Linear Attention Language Models Balance the Recall-Throughput Tradeoff," [arXiv:2402.18668](https://arxiv.org/abs/2402.18668), Theorem 3.1.

- ▶ **Idea (2023):** an RNN that trains in parallel like a Transformer and decodes with constant memory. Open source from the start.
- ▶ The lineage climbs the same ladder you just saw:

Version	State	Update rule
RWKV-4 (2023)	vector, $O(d)$	fixed per-channel decay
RWKV-5 and 6 (2024)	matrix	data-dependent decay (v6)
RWKV-7 “Goose” (2025)	matrix	generalised delta rule , vector learning rate

- ▶ RWKV-7 separates the key it *removes* from the key it *writes*. The authors prove it recognises all regular languages with a constant number of layers.
- ▶ **Status:** the largest RWKV-7 release is 2.9B. No frontier hybrid uses an RWKV mixer.

Peng et al., [arXiv:2305.13048](#) (RWKV-4), [arXiv:2404.05892](#) (Eagle and Finch), [arXiv:2503.14456](#) (RWKV-7). RWKV wiki, architecture history (Nov 2025).

- ▶ Hochreiter's group asks: how far do LSTMs go with modern scale and two repairs? They are **exponential gating** and a **matrix memory** (the mLSTM cell):

$$C_t = f_t C_{t-1} + i_t v_t k_t^\top$$

- ▶ Look familiar? It is gated linear attention plus an **input gate**. There is no erase term.
- ▶ **xLSTM 7B** (2025): trained on 2.3T tokens. About 50% faster generation than Mamba-based models, with quality the authors call comparable to similar-sized LLMs.
- ▶ A scaling study with 672 training runs (ICLR 2026) claims xLSTM is Pareto-better than Llama-style Transformers.

No xLSTM model above 7B exists, and adoption outside the originating lab is limited.

Beck et al., "xLSTM," NeurIPS 2024, [arXiv:2405.04517](#). "xLSTM 7B," [arXiv:2503.13427](#). "xLSTM Scaling Laws," ICLR 2026, [arXiv:2510.02228](#).

- ▶ **Hyena** (2023) mixes tokens with **long convolutions** whose filters come from a small network, interleaved with data-controlled gating. Cost is $O(L \log L)$.
- ▶ On text it matched small GPT models, but recall is its weak point. A 70M attention model beats a 1.4B Hyena on associative recall.
- ▶ It found its home where sequences are **long and untokenised**:

Evo 2 (Arc Institute, 2025): a 40B-parameter genome model trained on 9.3T DNA base pairs, with a context of 1M base pairs at single-nucleotide resolution. Of its 50 layers, 42 are convolutional and 8 are attention.

- ▶ Gu's argument: SSM-like mixers suit raw, high-resolution data. Attention suits tokens that already carry meaning.
- ▶ Even here, **a few attention layers stay**.

Poli et al., "Hyena Hierarchy," [arXiv:2302.10866](https://arxiv.org/abs/2302.10866). Arora et al., [arXiv:2312.04927](https://arxiv.org/abs/2312.04927). Arc Institute et al., "Genome modeling and design across all domains of life with Evo 2," bioRxiv (2025). Evo 2 40B configuration file, <https://github.com/ArcInstitute/evo2>.

Next-Generation Architectures: **Hybrid Architectures**

If a fixed state cannot recall and attention cannot stay cheap, why not use both?

- Keep the cheap recurrent mixer in most layers. Put full attention in a few.

1.3B, 100B tokens	Perplexity ↓	LM eval ↑	Retrieval ↑
Transformer++	16.9	50.9	41.8
DeltaNet	16.9	51.6	34.7
DeltaNet + attention in 2 layers	16.6	51.8	47.9

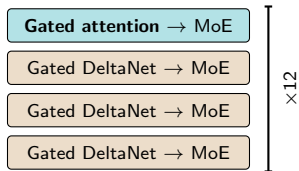
- A controlled study of 72 models confirms the shape. Language-modelling quality is **flat** in the mixing ratio. Recall **rises steeply** with more attention, and saturates near 3 recurrent layers per attention layer.
- The KV cache shrinks by the same factor, since only attention layers keep one.
- Surprise from the same study: a mixer's standalone quality does not predict its quality inside a hybrid.

Yang, "Linear Transformers for Efficient Sequence Modeling" (talk, 2024), slides 122 to 126. "A Systematic Analysis of Hybrid Linear Attention," [arXiv:2507.06457](https://arxiv.org/abs/2507.06457).

Model	Layer pattern	Total / active	Context
Qwen3.5 (Feb 2026)	3 Gated DeltaNet : 1 attention	397B / 17B	262K
Kimi K3 (2026)	69 KDA + 24 attention layers	2.8T / 104B	1M
Nemotron 3 Super (Mar 2026)	40 Mamba-2 + 8 attention	120.6B / 12.7B	1M
Jamba2 Mini (Jan 2026)	7 Mamba : 1 attention	52B / 12B	262K
Granite 4.0-H (Oct 2025)	9 Mamba-2 : 1 Transformer	32B / 9B	512K
Falcon-H1 (May 2025)	attention Mamba-2 in every block	up to 34B	256K

- ▶ Two camps: **delta-rule** mixers at about 3:1, and **Mamba-2** mixers with 8 to 20% attention layers.
- ▶ Read the ratio, not the label. Most of these models are also sparse MoEs.

Official model cards and configuration files (Qwen, Moonshot AI, NVIDIA, AI21, IBM, TII), 2025 to 2026. Raschka, “LLM Architecture Gallery: Hybrid Attention” (2026).



Qwen3-Next: 48 layers, 80B total, 3B active

- **Qwen3-Next** (Sep 2025) took Gated DeltaNet to production. Its makers claim $10\times$ the throughput of Qwen3-32B beyond 32K tokens. Qwen3.5 keeps the backbone.
- **Kimi Linear** ran the cleanest test: same data, same recipe, against a full-attention baseline.

	Kimi Linear	Full attention
MMLU-Pro (4K)	51.0	47.2
RULER (128K)	84.3	81.3
Decode at 1M, ms/token	1.84	11.48

- Serving had to change too. vLLM added a **hybrid cache manager** so a recurrent state and a KV block share the same memory pool.

Qwen3-Next-80B-A3B model card (2025). Kimi Team, “Kimi Linear: An Expressive, Efficient Attention Architecture,” [arXiv:2510.26692](https://arxiv.org/abs/2510.26692). vLLM blog, “vLLM Now Supports Qwen3-Next” (Sep 2025).

MiniMax

- ▶ **M1** (2025): 7 linear layers per softmax layer.
- ▶ **M2** (Oct 2025): **full attention in all 62 layers**.
- ▶ Their stated reasons:
 - deficits in multi-hop reasoning at scale that standard benchmarks missed
 - higher sensitivity to numerical precision
 - no mature prefix caching or speculative decoding for linear layers
- ▶ **M3** (2026): efficiency returns, but as *sparse* attention, not linear.

IBM

- ▶ **Granite 4.0-H** (Oct 2025): Mamba-2 hybrid.
- ▶ **Granite 4.1 and 4.2** (2026): back to a dense Transformer. The 8B dense model matches or beats the larger 4.0 hybrid.

Hybrids are a **contested** default, not a settled one.

MiniMax, "Why Did MiniMax M2 End Up as a Full Attention Model?" (2025). Lai et al., "MiniMax Sparse Attention," [arXiv:2606.13392](#). IBM, "Granite 4.1 LLMs: How They're Built" (Apr 2026).

- ▶ **Attention was not replaced.** What survived is a *layer type*. Recurrent layers carry most of the depth, and a few attention layers keep exact retrieval available.
- ▶ **Live designs:** delta-rule linear attention (Qwen, Kimi) and Mamba-2 (NVIDIA, AI21, TII). RWKV, xLSTM and Hyena keep niches.
- ▶ **Open problem 1, recall.** A fixed state has a capacity bound. Hybrids work around it. They do not solve it.
- ▶ **Open problem 2, state tracking.** Scalar decay cannot do parity or permutations. Mamba-3's rotations and RWKV-7's delta rule both attack this.
- ▶ **Open problem 3, systems.** Caching, quantisation and evaluation were all built for attention.

Next. The mixer is only half of the block. The other half, the feed-forward layer, holds most of the parameters.

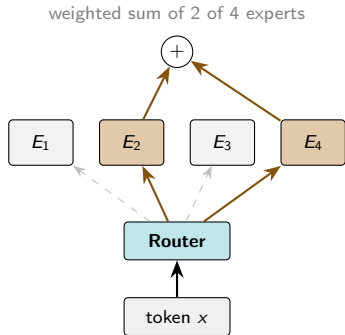
Next-Generation Architectures: **Mixture-of-Experts**

Must every token use every weight?

- ▶ In GPT-3, the feed-forward layers hold 116B of 175B parameters, about **two thirds**.
- ▶ MoE replaces one large FFN with N small *experts*. A **router** sends each token to only k of them:

$$y = \sum_{i \in \text{TopK}(x)} g_i(x) \text{FFN}_i(x)$$

- ▶ **Memory** scales with total parameters.
Compute scales with active parameters.



Parameter split: Gormley and Virtue, CMU 10-423 Lecture 16 (2025), slides 7 to 8. Gate equation after Jiang et al., “Mixtral of Experts,” [arXiv:2401.04088](https://arxiv.org/abs/2401.04088).

Model	Total / active	Active share	Experts	Top- <i>k</i>
Mixtral 8x7B (Dec 2023)	47B / 13B	28%	8	2
DeepSeek-V3 (Dec 2024)	671B / 37B	5.5%	256 + 1 shared	8
Qwen3-235B (Apr 2025)	235B / 22B	9.4%	128	8
Kimi K2 (Jul 2025)	1.04T / 32.6B	3.1%	384 + 1 shared	8
gpt-oss-120b (Aug 2025)	116.8B / 5.1B	4.4%	128	4
DeepSeek-V4-Pro (Apr 2026)	1.6T / 49B	3.1%	384 + 1 shared	6
Kimi K3 (2026)	2.8T / 104B	3.7%	896 + 2 shared	16

- **Sparser:** the active share fell from 28% to between 3 and 5% in about two and a half years.
- **Finer:** from 8 experts to hundreds. Why? Choosing 2 of 8 gives 28 combinations. Choosing 4 of 16 half-size experts gives 1,820, at the same active size.

Official configuration files and technical reports of each model. Combination count: Welleck, CMU 11-711 Lecture 21 (2026).
Kimi Team, "Kimi K2: Open Agentic Intelligence," [arXiv:2507.20534](https://arxiv.org/abs/2507.20534).

Strategy	How it works	Catch
Token choice (top- k)	each token picks its k best experts	load can be very uneven
Expert choice	each expert picks its best tokens	perfect balance, but leaks future tokens
Hash routing	fixed map from token id to expert	no learning at all
Soft MoE	experts process weighted mixes of tokens	not causal

- ▶ For a decoder the choice is forced. If an expert chooses among the tokens of a sequence, each decision depends on *later* tokens.
- ▶ So almost every LLM uses **token choice**, and must then solve load balancing separately.
- ▶ Old ideas return: DeepSeek-V4 uses *hash routing* in its first three layers.

Hashimoto, Stanford CS336 Lecture 4 (2026). Zhou et al., [arXiv:2202.09368](#). Wang et al., [arXiv:2408.15664](#). Puigcerver et al., [arXiv:2308.00951](#). DeepSeek-V4 report, [arXiv:2606.19348](#).

Softmax gate (Mixtral, Qwen3, gpt-oss)

$$g = \text{Softmax}(\text{TopK}(xW_g))$$

- ▶ Experts *compete*. Raising one score lowers the others.

- ▶ Sigmoid scoring pairs naturally with the bias-based balancing on the next slide. DeepSeek-V3, Kimi K2 and K3, and GLM-5 all use the pair.

- ▶ **2026:** DeepSeek-V4 changes the score again, to $\sqrt{\text{softplus}(\cdot)}$. The report gives no ablation for it.

Sigmoid gate (DeepSeek-V3, Kimi, GLM)

$$s_i = \sigma(x^\top e_i), \quad g_i = \frac{s_i}{\sum_{j \in \text{TopK}} s_j}$$

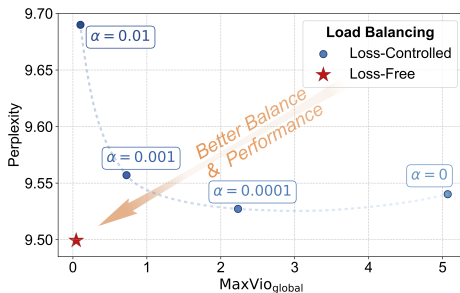
- ▶ Each expert is scored *on its own*, then the selected scores are normalised.

Jiang et al., [arXiv:2401.04088](#). DeepSeek-AI, “DeepSeek-V3 Technical Report,” [arXiv:2412.19437](#). DeepSeek-V4, [arXiv:2606.19348](#). Configuration files of Qwen3, gpt-oss, Kimi K2, Kimi K3 and GLM-5.

- ▶ **The dilemma.** A *balance loss* is an extra training term that penalises uneven use of the experts. It fights the language-modelling loss. Too weak: experts collapse. Too strong: quality drops.
- ▶ **The fix.** Add a per-expert bias b_i that is used *only to select*:

$$i \in \text{TopK}(\{s_j + b_j\}), \quad g_i \text{ still from } s_i$$

- ▶ After each step, nudge b_i up for underloaded experts and down for overloaded ones.
- ▶ No gradient ever flows from the balancing rule into the model.



Wang et al., “Auxiliary-Loss-Free Load Balancing Strategy for Mixture-of-Experts,” [arXiv:2408.15664](https://arxiv.org/abs/2408.15664), Figure 2. MaxVio measures how far the busiest expert is above the mean load.

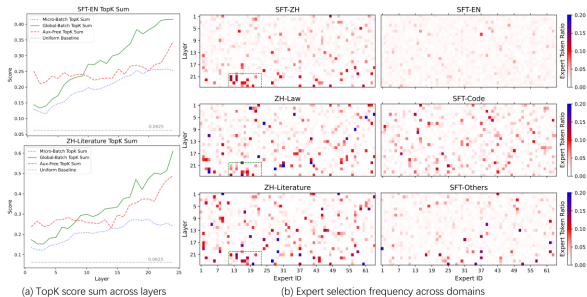
- Why did the bias method help? DeepSeek's own control experiment gives a surprising answer.

Validation loss	1B model	3B model
Auxiliary loss, balanced per <i>sequence</i>	2.258	2.085
Auxiliary loss, balanced per <i>batch</i>	2.253	2.080
Bias method (no auxiliary loss)	2.253	2.080

- The **scope** matters more than the method. Forcing every sequence to use all experts evenly stops experts from specialising. A code file *should* favour code experts.
- Qwen reached the same conclusion from the other side. Computing the balance loss over a **global batch** lifted a 43B model from 54.98 to 57.30 on MMLU.
- Note: frontier models still keep a very small sequence-level loss as a safety net.

DeepSeek-AI, [arXiv:2412.19437](#), Section 4.5.3. Qiu et al., "Demons in the Detail: On Implementing Load Balancing Loss for Training Specialized Mixture-of-Expert Models," [arXiv:2501.11873](#), Table 1.

Do Experts Specialise? An Updated Answer



- **2024 answer (Mixtral):** no clear domain specialisation. arXiv, biology and philosophy text route alike.
- **2025 answer:** it depends on training. With batch-level balancing, domain experts appear (above, right). OLMoE, trained from scratch, shows the same.

Figure: Qiu et al., [arXiv:2501.11873](#). Jiang et al., [arXiv:2401.04088](#), Section 5. Muennighoff et al., “OLMoE: Open Mixture-of-Experts Language Models,” [arXiv:2409.02060](#).

The Router Is Fragile: Stability and Token Dropping

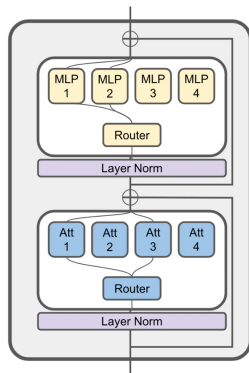
- ▶ Router logits pass through an exponential. In bfloat16, large logits mean large round-off, and training diverges.
- ▶ **Router z-loss** keeps the logits small:

$$L_z = \frac{1}{B} \sum_{i=1}^B \left(\log \sum_{j=1}^N e^{x_j^{(i)}} \right)^2$$

- ▶ The habit survives in 2026. Experts are stored in **4-bit** formats, while routers stay in higher precision.
- ▶ **Token dropping.** Early MoEs gave each expert a fixed capacity and dropped the overflow. Another user's request could cost you a token.
- ▶ Modern training is **dropless**. Block-sparse kernels let each expert take any number of tokens, and DeepSeek-V3 drops none in training or inference.

Zoph et al., "ST-MoE: Designing Stable and Transferable Sparse Expert Models," [arXiv:2202.08906](#). Gale et al., "MegaBlocks," [arXiv:2211.15841](#). DeepSeek-AI, [arXiv:2412.19437](#). Hashimoto, Stanford CS336 Lecture 4 (2026).

- ▶ **Question:** if routing works for the FFN, why not for attention heads?
- ▶ **Design:** every layer routes twice. Top-2 of 8 *attention experts*, then top-2 of 8 *MLP experts*. Keys and values are shared. Each attention expert owns its query and output projections.
- ▶ **Result:** 8B total, about 2B active. Trained on 1.25T tokens for **under \$0.1M**, and it beat Llama-2-7B on the Open LLM average (53.0 against 51.0).
- ▶ **Why it matters:** 30,000 H100 GPU-hours is a budget a university lab can reach.
- ▶ **What did not transfer:** none of the frontier configurations in this lecture routes its attention heads. Attention sparsity took a different road, which we meet in the next section.



Shen et al., "JetMoE: Reaching Llama2 Performance with 0.1M Dollars," [arXiv:2404.07413](https://arxiv.org/abs/2404.07413), Figure 1 and Table 3.

- **How fine should experts be?** Define granularity $G = d_{\text{ff}}/d_{\text{expert}}$. The fitted law

$$\mathcal{L}(N, D, G) = c + \left(\frac{g}{G^\gamma} + a \right) \frac{1}{N^\alpha} + \frac{b}{D^\beta}$$

says the compute-optimal G **grows with scale**, and $G = 1$ is almost never optimal.

- **How sparse?** At fixed compute, sparser is better, and the optimal sparsity approaches 1 as models grow.
- **The warning.** At matched loss, sparser models do **worse on reading comprehension**. Loss does not tell the whole story.

An old rule is dead: ST-MoE (2022) found the benefit “quickly diminishes” beyond 256 experts. Kimi K3 uses 896.

Krajewski et al., “Scaling Laws for Fine-Grained Mixture of Experts,” [arXiv:2402.07871](#). Abnar et al., [arXiv:2501.12370](#).
“Hyperparameter Scaling Laws Across MoE Sparsity,” [arXiv:2609.08690](#).

- ▶ **MoE is a throughput play at large batch.** At batch size 1 you pay memory for every expert and save only FLOPs.
- ▶ **Expert parallelism.** Experts live on different GPUs, so every MoE layer is an all-to-all exchange of tokens. DeepSeek serves V3 with experts spread over 144 GPUs in decode.
- ▶ **The batch must be large.** With 8 of 256 experts active, each expert sees few tokens unless many requests are pooled. In SGLang's tests, overlapping two batches in decode pays off only above 64 to 128 tokens per device.
- ▶ **Low-bit experts.** Experts are most of the weights, and each is used rarely. gpt-oss stores them at 4.25 bits per parameter, so the 120B model fits on one 80 GB GPU.
- ▶ **Co-design.** *LatentMoE* (NVIDIA, Kimi K3) projects each token to a smaller latent space before dispatch, and the experts compute there. The smaller payload pays for more experts and a larger top- k .

DeepSeek, "DeepSeek-V3/R1 Inference System Overview" (2025). SGLang team, large-scale expert parallelism blog (May 2025). OpenAI, gpt-oss model card, [arXiv:2508.10925](https://arxiv.org/abs/2508.10925). "LatentMoE," [arXiv:2601.18089](https://arxiv.org/abs/2601.18089).

- ▶ The newest frontier MoEs activate **3 to 5%** of their weights per token, across hundreds of small experts.
- ▶ Routing is **token choice**, because a decoder must stay causal.
- ▶ Balancing moved from a loss term to a **selection bias**, and from the sequence to the **batch**. That shift is what lets experts specialise.
- ▶ The router is the delicate part: keep its logits small and its precision high.
- ▶ Open design questions: shared experts, dense early layers, and the right score function.

Next. We made the feed-forward block sparse. Can we do the same to attention itself, without giving up exact recall?

Next-Generation Architectures: **Efficient Attention**

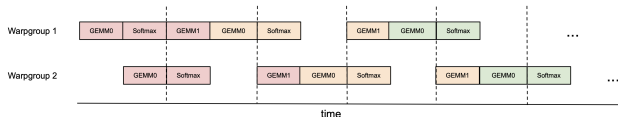
Can we keep attention, and its exact recall, but pay less for it?

Idea	What it changes	Still exact attention?
Better kernels	how the same maths runs on the GPU	yes
Local windows	each token sees only its neighbours	no, long range is cut
Learned sparsity	each token sees a <i>chosen</i> subset	no, but the model picks
Context parallelism	one sequence is split across GPUs	yes

- ▶ The first idea dates from 2022. **FlashAttention** computes exact attention in tiles that fit in fast on-chip memory, and never writes the $L \times L$ matrix out.
- ▶ That fixed the bottleneck of its day, which was *memory traffic*. The hardware then moved on. So did the bottleneck.

FlashAttention-3: When Softmax Became the Slow Part

- ▶ On an H100, matrix multiply runs at **989 TFLOPS**. Special functions such as exp run at **3.9**. FlashAttention-2 reaches only about 35% of the GPU's peak.
- ▶ **Idea 1, overlap.** Two groups of threads take turns: while one does the softmax, the other runs matrix multiplies. The slow step hides under the fast one.



- ▶ **Idea 2, FP8.** Halve the precision, double the throughput. Per-block scaling and a random rotation of Q and K keep the error $2.6\times$ below naive FP8.
- ▶ **Result:** 1.5 to $2\times$ faster than FlashAttention-2, up to 740 TFLOPS (75% of peak).

Shah et al., "FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision," [arXiv:2407.08608](https://arxiv.org/abs/2407.08608), Figure 1 and Table 3. Dao, FlashAttention-3 blog (2024).

- ▶ From H100 to B200, tensor-core throughput grew about $2.25\times$. The units that compute exp, and shared-memory bandwidth, **did not grow**.
- ▶ So the exponential is now a co-bottleneck in the forward pass. FlashAttention-4 answers with algorithm changes, not only scheduling:
 - compute part of the exp calls in *software*, with a cubic polynomial on idle arithmetic units
 - rescale the running softmax only when the maximum jumps
 - keep intermediate tiles in the new on-chip tensor memory
- ▶ **Result:** 1613 TFLOPS on a B200 (71% of peak), up to $1.3\times$ over cuDNN.

Each GPU generation moves the binding resource: memory traffic, then thread scheduling, then exp throughput. A kernel is never finished.

Zadouri et al., "FlashAttention-4: Algorithm and Kernel Pipelining Co-Design for Asymmetric Hardware Scaling," [arXiv:2603.05451](#). Dao, FlashAttention-4 blog (2026).

- ▶ Kernels do not shrink the KV cache. The simplest way to do that: **most layers look only at recent tokens.**

Model	Pattern (local : global layers)	Window
Mistral 7B (2023)	all local	4096
Gemma 2 (2024)	1 : 1	4096
Gemma 3 (2025)	5 : 1	1024
gpt-oss (2025)	1 : 1, with learned attention sinks	128
Llama 4 (2025)	3 : 1, local layers attend within fixed chunks	8192

- ▶ The trend: **windows shrink, and a few global layers carry the long range.**
- ▶ Gemma 3: KV memory overhead at 32K tokens falls from about 60% to under 15%.
- ▶ The same shape as the hybrids. Cheap layers do most of the work, and a few expensive ones keep the capability.

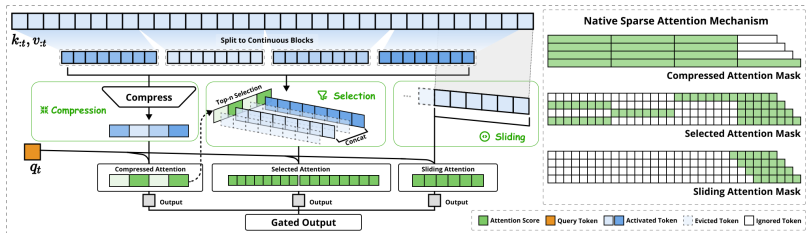
Jiang et al., [arXiv:2310.06825](#). Gemma Team, [arXiv:2408.00118](#) and [arXiv:2503.19786](#), Figure 5. OpenAI, [arXiv:2508.10925](#). Hugging Face, Llama 4 release blog (2025).

- ▶ **A common misreading.** Stacking k layers of window W gives a theoretical span of $W \times k$. That is a receptive field, not reliable long-range recall.
- ▶ **Evidence.** In MiniMax's ablation, full attention scores 90 on RULER at 128K. The sliding-window variant scores 72.
- ▶ **Memory is not latency.** Mistral dropped sliding windows in a later model: they reduce memory but do not necessarily reduce inference latency.
- ▶ Fixed patterns were tried long ago (Sparse Transformer, Longformer, BigBird, 2019 to 2020). Only the local window survived.

A fixed pattern decides where to look before reading the input. What if the model *chose* which distant tokens matter?

Raschka, "MiniMax M2 Technical Report Notes" (May 2026) and "The Big LLM Architecture Comparison" (2025, updated 2026). Child et al., [arXiv:1904.10509](#). Beltagy et al., [arXiv:2004.05150](#). Zaheer et al., [arXiv:2007.14062](#).

Native Sparse Attention: Three Views of the Past



- ▶ Each query attends through three branches, mixed by learned gates: **compressed** blocks (a coarse view of everything), **selected** blocks (the top- n most relevant, at full detail), and a **sliding window**.
- ▶ The compressed scores are reused to choose the blocks, so selection is almost free.
- ▶ Trained sparse from the start. At 64K context: $11.6\times$ faster decoding and $9\times$ faster forward pass, with benchmark scores at or above the full-attention baseline.

Yuan et al., "Native Sparse Attention: Hardware-Aligned and Natively Trainable Sparse Attention," [arXiv:2502.11089](https://arxiv.org/abs/2502.11089), Figure 2.
Reported by the authors and not yet independently replicated.

- ▶ A tiny **lightning indexer** scores every past token for the current query:

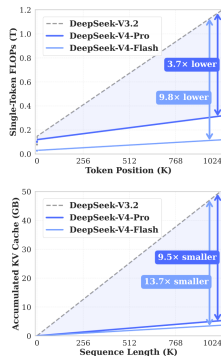
$$l_{t,s} = \sum_{j=1}^{H^l} w_{t,j}^l \text{ReLU}(q_{t,j}^l \cdot k_s^l)$$

- ▶ Only the **top 2048** tokens go to the real attention. Its cost falls from $O(L^2)$ to $O(Lk)$.
- ▶ **It can be retrofitted.** DeepSeek converted a dense checkpoint: a short warm-up where the indexer imitates dense attention, then sparse training. GLM-5 repeated the recipe with only 20B tokens.
- ▶ **Impact:** DeepSeek cut API prices by more than 50% with V3.2.

Contrast with Part 1. Hybrids *compress* the past into a state. Sparse attention keeps every token and *retrieves*.

DeepSeek-AI, “DeepSeek-V3.2,” [arXiv:2512.02556](https://arxiv.org/abs/2512.02556). DeepSeek API news (Sep 2025). Zhipu AI, “GLM-5,” [arXiv:2602.15763](https://arxiv.org/abs/2602.15763).

- ▶ One problem remained: the indexer still scans every token. V4 shrinks the sequence *before* scoring.
- ▶ Two layer types alternate through the network:
 - **CSA**: compress keys and values $4\times$, then keep the top- k compressed entries. A sparse, fine view.
 - **HCA**: compress $128\times$ and attend to all of it. A dense, coarse view.
- ▶ Every layer adds a window of 128 recent uncompressed tokens.
- ▶ At 1M tokens, a CSA layer in V4-Pro reads about 1,150 entries. An HCA layer reads about 8,300.



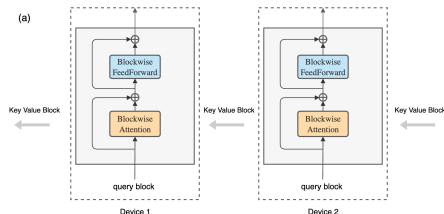
DeepSeek-AI, "DeepSeek-V4," [arXiv:2606.19348](https://arxiv.org/abs/2606.19348), Figure 1 (right) and Section 2.3. Entry counts derived from the published hyperparameters ($k = 1024$, window 128, $2^{20}/128$).

- ▶ **November 2025.** MiniMax ships M2 with full attention and writes that no efficient variant has shown stable superiority in production.
- ▶ **June 2026.** MiniMax ships M3 with its own block-sparse attention: $14.2\times$ faster prefill and $7.6\times$ faster decoding at 1M tokens, on par with the dense baseline.
- ▶ **What changed?** Not the idea. The *kernels* (selection matched to GPU blocks) and the *training recipes* (adapt from a dense checkpoint).
- ▶ The frontier in mid-2026 is the indexer itself. GLM-5.2 reuses one indexer result across four layers and removes 75% of the indexer's compute.
- ▶ No training budget? **Training-free** methods (Quest, MInference, DuoAttention) select KV entries at inference time on an unmodified model.

MiniMax, “No Free Lunch” blog (LMSYS, Nov 2025). Lai et al., [arXiv:2606.13392](#). Bai et al., “IndexCache,” [arXiv:2603.12201](#). Tang et al., [arXiv:2406.10774](#).

Ring Attention: One Sequence, Many GPUs

- ▶ Everything so far saves compute on *one* device. Training on million-token sequences needs the opposite: **split one sequence across devices**.
- ▶ Each device keeps its own block of queries. Key and value blocks **travel around a ring**.
- ▶ FlashAttention's blockwise softmax makes the order of arrival irrelevant, so the result is exact.
- ▶ Sending a block overlaps with computing on the previous one. Communication is free if each block is large enough: about 1,000 tokens on an A100.
- ▶ Memory per device no longer depends on total sequence length.



Liu, Zaharia, Abbeel, "Ring Attention with Blockwise Transformers for Near-Infinite Context," [arXiv:2310.01889](https://arxiv.org/abs/2310.01889), Figure 2.

- ▶ **Problem with a plain ring under a causal mask:** the device holding the last tokens has far more work than the one holding the first. *Striped Attention* interleaves tokens across devices to even the load.
- ▶ **What Meta did for Llama 3.** No ring at all. Each device simply gathers all keys and values. Two reasons: with grouped-query attention the KV tensors are small, and the gather is $O(S)$ under an $O(S^2)$ computation.

Setup	Sequence length	GPU utilisation (MFU)
Llama 3, 16,384 GPUs, 16-way context parallel	131,072	38%
torchTitan, Llama3-8B on 32 H100, all-gather	1M	25.7%
torchTitan, same, ring-style all-to-all	1M	12.1%

- ▶ **Open question:** how to split learned sparse attention, where the tokens a query needs are not known in advance.

Brandon et al., [arXiv:2311.09431](https://arxiv.org/abs/2311.09431). Meta, “The Llama 3 Herd of Models,” [arXiv:2407.21783](https://arxiv.org/abs/2407.21783), Table 4. PyTorch forum, “Training long-context LLMs with 1M sequence length using context parallel” (Jan 2025).

- ▶ **Kernels** (FlashAttention-3 and 4) give exact attention at 70 to 75% of GPU peak. They follow the hardware, generation by generation.
- ▶ **Windows** cut the KV cache cheaply, but a few global layers must remain.
- ▶ **Learned sparse attention** became a production default within 2025 to 2026: a cheap scorer, a top- k selection, and a local window.
- ▶ **Context parallelism** is how million-token sequences are trained at all.

Next. We can now afford a million tokens. Affording them and *using* them are different problems.

Next-Generation Architectures: **Long Context**

We can now afford a million tokens. Can the model actually use them?

1. **Positions.** The model meets position indices it never saw in training. With RoPE, that means rotation angles outside the trained range.
2. **Attention itself.** Trained heads rely on habits, such as dumping spare attention on the first token. Change the window and those habits fail.
3. **Finding things.** Even when nothing breaks, retrieving the right fact among a million tokens is hard. And we need honest ways to measure it.

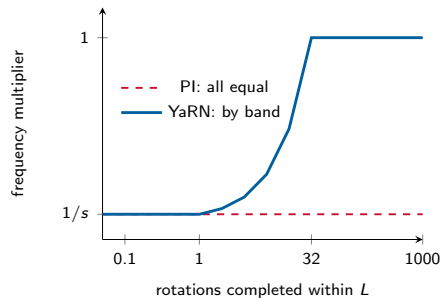
This section follows that order: **extend the positions, tame the sinks, add memory, then train and evaluate** at 1M tokens.

Why RoPE Fails Beyond Its Training Length

- ▶ RoPE rotates each pair of dimensions at its own frequency. Dimension d has wavelength

$$\lambda_d = 2\pi b^{2d/|D|}$$

- ▶ **Fast** dimensions complete many rotations inside the training length L . They have seen every angle.
- ▶ **Slow** dimensions never finish one rotation. Past L they produce angles the model has **never seen**.
- ▶ **Position Interpolation:** squeeze new positions into the old range, $m \mapsto mL/L'$. It works after a short fine-tune, but it also squeezes the fast dimensions, which blurs *local* order.



Schematic from the YaRN ramp with $\alpha = 1$, $\beta = 32$,
 $s = 8$.

Chen et al., "Extending Context Window of Large Language Models via Positional Interpolation," [arXiv:2306.15595](#). Peng et al., "YaRN," [arXiv:2309.00071](#).

- ▶ Let $r(d) = L/\lambda_d$ be the rotations a dimension completes in training. YaRN blends per dimension:

$$\theta'_d = (1 - \gamma(r)) \frac{\theta_d}{s} + \gamma(r) \theta_d, \quad \gamma(r) = \begin{cases} 0 & r < \alpha \\ 1 & r > \beta \\ \frac{r - \alpha}{\beta - \alpha} & \text{otherwise} \end{cases}$$

- ▶ **Fast dimensions** are left alone. **Slow dimensions** are fully interpolated. The rest are blended.
- ▶ Longer contexts also flatten attention, so YaRN adds a temperature, $\sqrt{1/t} = 0.1 \ln s + 1$.
- ▶ **Cost:** about 400 fine-tuning steps, against 1,000 for Position Interpolation.

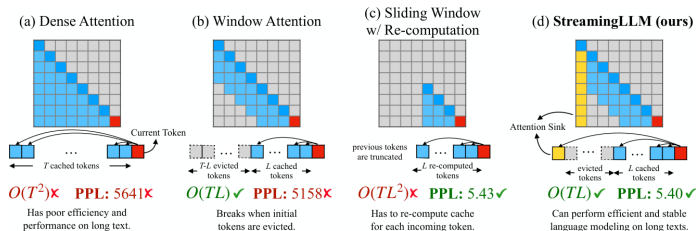
Still in use in 2026. gpt-oss, Qwen3-Next and DeepSeek-V4 all ship with YaRN scaling in their configurations.

Peng et al., "YaRN: Efficient Context Window Extension of Large Language Models," [arXiv:2309.00071](https://arxiv.org/abs/2309.00071). Model cards and configurations of gpt-oss, Qwen3-Next and DeepSeek-V4.

- ▶ **Search it.** *LongRoPE* finds a separate scale for every dimension by evolutionary search, and reaches 2M tokens. *LongRoPE2* adds one insight: slow dimensions are under-trained, so the cut-off sits earlier than theory says. On Llama-3-8B at 128K it scores 82.0 on RULER with **10B** tokens of training. Meta used 800B.
- ▶ **Train it.** Frontier labs now build extension into pre-training. They raise the RoPE base (Llama 3 uses 500,000) and lengthen sequences in stages.
- ▶ **Remove it.** A growing group uses **no positional encoding** in the global-attention layers, and leaves order to local or recurrent layers:
 - Llama 4 interleaves NoPE layers with RoPE layers
 - Nemotron 3 and Kimi Linear: the recurrent layers already encode order
 - DeepSeek-V4 and Qwen3-Next apply RoPE to only a fraction of the dimensions

Ding et al., “LongRoPE,” [arXiv:2402.13753](#). Shang et al., “LongRoPE2,” [arXiv:2502.20082](#), Table 2. Meta, [arXiv:2407.21783](#) and Llama 4 blog (2025). NVIDIA, [arXiv:2512.20856](#).

Attention Sinks: Why Evicting the First Token Breaks the Model



- ▶ Trained LLMs send a large share of attention to the **first few tokens**, whatever they contain. Evict them from a rolling cache and perplexity explodes (panel b).
- ▶ **StreamingLLM:** always keep 4 sink tokens, plus a rolling window. Stable for more than 4M tokens, with no fine-tuning.
- ▶ A caution from its authors: endless chatting is *not* infinite context. Evicted tokens are gone.

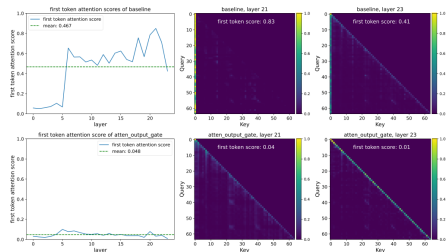
Xiao et al., "Efficient Streaming Language Models with Attention Sinks," ICLR 2024, [arXiv:2309.17453](https://arxiv.org/abs/2309.17453), Figure 1. Han, MIT 6.5940 Lecture 15 (2024).

Why Sinks Form, and Two Ways to Engineer Them

- **Cause.** Softmax weights must sum to 1. A head with nothing useful to say still has to attend *somewhere*. The first token becomes its “do nothing” slot.
- Sinks get **stronger** with longer training context and larger models.
- **Fix A: give it a real null slot.** Add a learned term to the denominator (gpt-oss, DeepSeek-V4):

$$a_{ij} = \frac{e^{z_{ij}}}{\sum_k e^{z_{ik}} + e^{z'}}$$

- **Fix B: let the head switch off.** Gate the attention output, $Y' = Y \odot \sigma(XW_{\theta})$ (Qwen).



With output gating, mean first-token attention falls from 0.467 to 0.048.

Barbero et al., “Why do LLMs attend to the first token?” [arXiv:2504.02732](#). DeepSeek-V4, [arXiv:2606.19348](#), Eq. 27. Figure: Qiu et al., “Gated Attention for Large Language Models,” [arXiv:2505.06708](#).

- ▶ A different answer to long context: keep the window small, and add a **separate memory** for what falls outside it.

Model	What the memory is	Reach
Transformer-XL (2019)	cached hidden states of the previous segment	a few segments
Compressive Transformer (2019)	old states, compressed instead of dropped	longer, lossy
Memorizing Transformer (2022)	nearest-neighbour lookup over past keys and values	262K tokens
Recurrent Memory Transf. (2023)	memory tokens passed between segments	2M (synthetic)
Titans (2025)	a small MLP trained at inference time	over 2M (claimed)

- ▶ The common shape: **exact attention nearby, a lossy channel for the distant past.**
- ▶ That lossy channel is the fixed-size state of Part 1 again, with the same capacity limit.

Dai et al., [arXiv:1901.02860](#). Rae et al., [arXiv:1911.05507](#). Wu et al., [arXiv:2203.08913](#). RMT, [arXiv:2304.11062](#). Behrouz et al., [arXiv:2501.00663](#).

Titans: memorise what surprises you

- ▶ The memory $\mathcal{M}$ is an MLP. At each token it takes a gradient step on $\|\mathcal{M}(k_t) - v_t\|^2$.
- ▶ The gradient is the *surprise*. Momentum carries it over the next tokens. Weight decay forgets.
- ▶ It is the delta rule of Part 1, with a deep memory in place of a matrix.

No independent replication at 7B or above. Its successor's own paper finds Transformers still best at in-context recall.

Infini-attention: a cautionary tale

- ▶ Google (2024): add a compressive memory to each attention head. Claim: a 1B model solves passkey retrieval at 1M tokens.
- ▶ The memory is one linear-attention state per head.
- ▶ Hugging Face tried to reproduce it on Llama 3 8B. The memory gates stayed near 0.5, and needles in earlier segments were **not found**.

No code or weights were released. Treat un-replicated long-context claims with care.

Behrouz, Zhong, Mirrokni, "Titans," [arXiv:2501.00663](#). Behrouz et al., "ATLAS," [arXiv:2505.23735](#). Munkhdalai et al., [arXiv:2404.07143](#). Hugging Face blog, "A failed experiment: Infini-Attention" (2024).

How Models Are Actually Trained to 1M Tokens

Model	Length schedule	Tokens spent on long stages
Llama 3 (2024)	6 stages, 8K to 128K	about 800B
MiniMax-01 (2025)	128K, then 512K, then 1M	300B + 32B + 26B
Nemotron 3 Super (2026)	8K, then straight to 1M	34B + 17B
DeepSeek-V4 (2026)	4K, 16K, 64K, then 1M	not reported per stage

- ▶ **The last stage is cheap.** A few tens of billions of tokens at 1M, after trillions at short length.
- ▶ **Gate each stage.** Llama 3 moved on only when short-context quality had recovered *and* needle retrieval was perfect at the new length.
- ▶ **Needle tests saturate early.** MiniMax found them solved long before real long-context ability appeared, and tracked harder tasks instead.

Meta, [arXiv:2407.21783](#). MiniMax, [arXiv:2501.08313](#), Table 6. NVIDIA, “Nemotron 3 Super,” [arXiv:2604.12374](#). DeepSeek-AI, [arXiv:2606.19348](#).

- ▶ Each new benchmark asked a harder question, and each one **lowered** the measured usable context.

Benchmark	What it asks	What it found
Needle in a haystack (2023)	find one planted sentence	most models now near perfect
Lost in the Middle (2023)	answer from one of 20 documents	accuracy is U-shaped in position: 75.8% first, 53.8% middle
RULER (2024)	multi-hop tracing, aggregation	of 17 models claiming 32K+, about half are satisfactory at 32K
NoLiMa (2025)	needles with no word overlap with the question	GPT-4o falls from 99.3% to 69.7% at 32K

- ▶ NoLiMa's "effective length" for GPT-4o is **8K** tokens. Literal matching is easy. **Reasoning over** a long context is what degrades.

Liu et al., [arXiv:2307.03172](https://arxiv.org/abs/2307.03172). Hsieh et al., "RULER," [arXiv:2404.06654](https://arxiv.org/abs/2404.06654). Modarressi et al., "NoLiMa," [arXiv:2502.05167](https://arxiv.org/abs/2502.05167).

The 2026 Picture: What Happens Between 128K and 1M

Model	to 128K	to 1M	Decay
Claude Opus 4.6	77.1	70.6	8.5%
Gemini 3.1 Pro	77.8	68.5	12.0%
GPT-5.5	74.6	67.8	9.2%
DeepSeek-V4-Pro	71.6	59.1	17.4%
Qwen3.5-397B	71.0	54.4	23.3%
GLM-5	64.3	44.3	31.1%

ATLAS score, averaged over lengths from 8K up to the stated limit.

- ▶ An **independent** harness over 26 models: the average score drops **24.3%** when the range extends from 128K to 1M.
- ▶ Retrieval and question answering lose the most at 1M.
- ▶ Closed frontier models decay by 8 to 14%. Open models by 17 to 60%.
- ▶ **Beware vendor tables.** Labs report the same benchmark in different formats, and their numbers for each other's models disagree.

A 1M-token window is real. Uniform quality across that window is not there yet.

Huang et al., "ATLAS," [arXiv:2605.28079](https://arxiv.org/abs/2605.28079), Tables 10 and 11 (May 2026).

- ▶ **Positions:** scale RoPE by frequency band (YaRN), or drop positional encoding from global layers altogether.
- ▶ **Sinks:** softmax needs a “do nothing” option. Modern models provide one on purpose.
- ▶ **Memory modules** trade exact recall for reach. Their strongest claims are not yet replicated at scale.
- ▶ **Training:** stage the length, gate each stage, and spend little at the longest length.
- ▶ **Evaluation:** trust independent, reasoning-heavy benchmarks over needle tests and vendor tables.

Next. Every design so far changes what a parameter or a token costs. How should a fixed budget be divided between model size, data and inference?

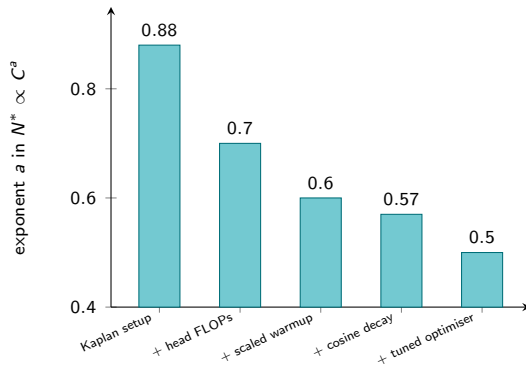
Next-Generation Architectures: **Scaling Laws and Their Limits**

Given a fixed budget, how big should the model be, and how long should it train?

- ▶ **Recap.** Loss follows a power law in parameters N and tokens D . For a compute budget $C \approx 6ND$ there is an optimal split.
 - Kaplan et al. (2020): grow the model faster, $N^* \propto C^{0.73}$.
 - Chinchilla (2022): grow both equally, $N^* \propto C^{0.5}$. About **20 tokens per parameter**.
- ▶ **Correction 1.** Chinchilla's own parametric fit was wrong. Taken literally it implies about 70 tokens per parameter. A careful refit recovers 20, and shows the reported confidence intervals were implausibly tight.
- ▶ **Correction 2.** Kaplan and Chinchilla never disagreed about Transformers. The gap is a *measurement artefact*, as the next slide shows.

“20 tokens per parameter” is a property of one training setup. It is a useful heuristic, not a law.

Hoffmann et al., [arXiv:2203.15556](#). Besiroglu et al., “Chinchilla Scaling: A replication attempt,” [arXiv:2404.10102](#). Pearce and Song, [arXiv:2406.12907](#).



- ▶ Over 900 training runs reproduce Kaplan's exponent, then remove it step by step.
- ▶ The biggest single cause: **not counting the compute of the output layer**, which matters at small scale.
- ▶ A side result against Chinchilla: learning-rate decay is *not* needed for the 0.5 law.
- ▶ **Lesson for your own fits:** scaling exponents are fragile to protocol. Small models and untuned hyperparameters bias them.

Porian et al., "Resolving Discrepancies in Compute-Optimal Scaling of Language Models," [arXiv:2406.19146](https://arxiv.org/abs/2406.19146), Figure 1.

Model	Tokens / param
GPT-3	2
Chinchilla	20
Llama 2 70B	29
Mistral 7B	110
Llama 3 70B	215
Llama 3 8B	about 1,900

Last row derived from 15T tokens and 8B parameters.

- ▶ Chinchilla minimises **training** compute for a target loss. A deployed model also pays for every token it ever generates.
- ▶ If inference demand is large, the best choice is a **smaller model trained far longer**. Quality keeps improving up to 10,000 tokens per parameter.
- ▶ Meta's own account: the Chinchilla-optimal budget for an 8B model is about 200B tokens. Llama 3 8B kept improving log-linearly up to 15T.
- ▶ Flagships stay near compute-optimal. **Small models are over-trained by 100× or more.**

Table: Hashimoto, Stanford CS336 Lecture 9 (2026). Sardana et al., "Beyond Chinchilla-Optimal," [arXiv:2401.00448](https://arxiv.org/abs/2401.00448). Meta, "Introducing Meta Llama 3" (Apr 2024).

- ▶ Scaling laws assume fresh data. What if you must repeat it?
- ▶ **Repetition has a budget.** Up to about 4 epochs is almost as good as new data. The value of further epochs decays exponentially:

$$D' = U_D + U_D R_D^* (1 - e^{-R_D/R_D^*}), \quad R_D^* \approx 15$$

where U_D is unique tokens and R_D the number of repetitions.

- ▶ **The stock is finite.** Epoch AI estimates about 300T tokens of usable public text, fully used some time between 2026 and 2032.
- ▶ **Responses:** stronger regularisation, synthetic data for maths and code, and compute spent on reinforcement learning instead of more text.

Sutskever, NeurIPS 2024: “We’ve achieved peak data and there’ll be no more. There’s only one internet.”

Muennighoff et al., “Scaling Data-Constrained Language Models,” [arXiv:2305.16264](#). Villalobos et al., “Will we run out of data?” Epoch AI (2024). Kim et al., [arXiv:2509.14786](#). Quote as reported by The Verge (13 Dec 2024).

Study	What was done	Effect
FineWeb-Edu (2024)	keep web pages an LLM rates as educational	MMLU 33 to 37%, ARC 46 to 57%. Matches the baseline with about $10\times$ fewer tokens
DCLM (2024)	a simple fastText quality classifier over a 240T-token pool	comparable to Llama 3 8B on MMLU with about $6.6\times$ less compute
Data mixing laws (2025)	predict loss as a function of mixture proportions	the optimised mix matches the default mix trained 48% longer

- ▶ Classical scaling laws have no term for quality. In practice it shifts the whole curve.
- ▶ **The tension:** aggressive filtering shrinks the pool of unique tokens, which brings back the repetition problem of the previous slide.
- ▶ The best filter depends on the budget. With more compute, filter less.

Penedo et al., "The FineWeb Datasets," [arXiv:2406.17557](#). Li et al., "DataComp-LM," [arXiv:2406.11794](#). Ye et al., "Data Mixing Laws," [arXiv:2403.16952](#). Goyal et al., [arXiv:2404.07177](#).

- ▶ Kaplan's finding still holds at scale: for a fixed parameter count, loss depends only weakly on shape. So shape is set by convention and by hardware.

Choice	Convention	Recent outliers
FFN width / model width	4, or 8/3 with gated units	Gemma 2 uses 8
Model width / depth	100 to 200	Gemma 3: 87. Gemma 4: 61
Heads $\times$ head dim / width	1	Qwen 3.5: 1.2
Vocabulary	100K to 250K in production	Gemma 4: 262K

- ▶ **Where shape does matter:**
 - *Small models.* Below a billion parameters, deep and thin beats shallow and wide (MobileLLM).
 - *Vocabulary.* The optimal size grows with compute. By one estimate, Llama 2 70B's 32K vocabulary should have been at least 216K.

Table: Hashimoto, Stanford CS336 Lecture 3 (2026). Liu et al., "MobileLLM," [arXiv:2402.14905](#). Tao et al., "Scaling Laws with Vocabulary," [arXiv:2407.13623](#).

- **Precision.** Treat bit-width as a third resource. Low precision acts like a reduction in effective parameters:

$$N_{\text{eff}} = N (1 - e^{-P_w/\gamma_w}) (1 - e^{-P_a/\gamma_a}) (1 - e^{-P_{kv}/\gamma_{kv}})$$

Two consequences. About 7 to 8 bits is compute-optimal for training. And **over-trained models quantise worse**, which is exactly the small deployed models of two slides ago.

- **Architecture.** Hybrids, MoE and multimodal models get their own fitted laws. The exponents stay close to the dense Transformer's.
- **Emergence.** Are sudden jumps in ability real? Two answers that fit together:
 - with continuous metrics, most “emergent” curves become smooth
 - abilities appear when the *pre-training loss* crosses a threshold, whatever the model size
- So the working recipe is two-stage: predict loss from compute, then task score from loss.

Kumar et al., “Scaling Laws for Precision,” [arXiv:2411.04330](#). Schaeffer et al., [arXiv:2304.15004](#). Du et al., [arXiv:2403.15796](#). MiniMax, [arXiv:2501.08313](#).

- ▶ Training laws trade training FLOPs for loss. Since 2024 there is a second curve: **inference FLOPs for task success**.
- ▶ OpenAI's o1 on AIME 2024 mathematics problems:

Setting	Problems solved
GPT-4o	12%
o1, one sample	74%
o1, majority vote over 64 samples	83%
o1, 1,000 samples re-ranked by a learned scorer	93%

- ▶ At matched FLOPs, a small model with well-allocated test-time compute can beat a model **14× larger**.
- ▶ The catch: gains depend on a **verifier**. Without one, voting saturates after a few hundred samples.

OpenAI, "Learning to reason with LLMs" (Sep 2024). Snell et al., [arXiv:2408.03314](https://arxiv.org/abs/2408.03314). Brown et al., "Large Language Monkeys," [arXiv:2407.21787](https://arxiv.org/abs/2407.21787).

- ▶ **Compute keeps growing.** Frontier training compute has risen about $5\times$ per year since 2020. The largest estimated run so far is Grok 4, at about 5×10^{26} FLOP.
- ▶ **What binds next is power, then data.** Epoch AI projects runs near 2×10^{29} FLOP as feasible by 2030, with electricity the first hard ceiling.
- ▶ **Pre-training alone pays less.** GPT-4.5 used roughly ten times GPT-4's compute for incremental gains. GPT-5 then used *less* training compute than GPT-4.5: scaling post-training on a smaller model gave better returns.
- ▶ **Reinforcement learning has its own curve.** It is sigmoidal in compute, not a power law. Most recipe choices change how fast you approach the ceiling. Few change the ceiling.

Be careful with “scaling has hit a wall”. No lab has published a frontier-scale loss curve that shows the power law breaking.

Epoch AI, trends dashboard (Feb 2026), “Can AI scaling continue through 2030?” (2024) and “Why GPT-5 used less training compute than GPT-4.5” (Sep 2025). Lambert, Interconnects (Feb 2025). Meta, “The Art of Scaling RL Compute for LLMs,” [arXiv:2510.13786](https://arxiv.org/abs/2510.13786).

- ▶ The compute-optimal rule survives, but **its constants depend on protocol**. Fit your own.
- ▶ Deployed models are deliberately **over-trained**, because inference dominates lifetime cost.
- ▶ **Data** is now a binding constraint: its quantity, its quality and how often it can be repeated.
- ▶ Compute has three destinations: **pre-training, post-training and inference**. Each has a different curve.

Next. All of this assumed text. The last section asks what changes when the model must also see, hear and draw.

Next-Generation Architectures: **Multimodal Foundation Models**

What changes when the model must also see, hear and draw?

Recap: Where Do Other Modalities Enter the Model?

Fusion	Mechanism	Examples
Mid-fusion	a separate encoder produces features, and a projector injects them into the LLM	LLaVA, Qwen3-VL, InternVL
Early fusion	one operator maps every modality into the same token space, and one backbone processes all	Chameleon, Transfusion

- ▶ The question has moved from *where* to fuse to *when in training* to fuse, and *how to share capacity* between modalities.
- ▶ The output side opened up too. Models now generate images, speech and video.

Mind the vocabulary. Qwen calls joint multimodal pre-training “early fusion” even with a separate encoder. The survey definitions above would call the same model mid-fusion.

An et al., “Toward Native Multimodal Modeling: A Roadmap,” [arXiv:2605.25343](https://arxiv.org/abs/2605.25343). Qwen3.5 model card (2026).

Model	Backbone	How vision is built in
Kimi K2.5 (Jan 2026)	MoE, 1.04T / 32B active	a 400M native-resolution ViT, with 15T tokens of joint vision and text pre-training
Qwen3.5 (Feb 2026)	3:1 hybrid MoE, 397B / 17B	one checkpoint replaces the separate Qwen3-VL line
Gemma 4 (Jul 2026)	2.3B to 31B, dense and MoE	a 550M ViT with selectable budgets of 70 to 1,120 tokens per image

- ▶ Until 2025, most labs shipped a text model and a separate “-VL” variant. Each of these three ships **one natively multimodal checkpoint**.
- ▶ Note the backbones. They are the hybrid and MoE designs from earlier today. The novelty is in the backbone and in token budgets, not in a new fusion trick.

Kimi Team, “Kimi K2.5,” [arXiv:2602.02276](#). Qwen3.5-397B-A17B model card. Gemma Team, “Gemma 4,” [arXiv:2607.02770](#).

A vendor ablation (Kimi K2.5)

- ▶ Fixed total budget of vision and text tokens.
- ▶ Adding vision **early, at only 10%** of the mix, beats adding it **late at 50%**.
- ▶ The ratio matters little. The timing matters.
- ▶ A bonus: reinforcement learning on *vision* tasks also raised *text* scores, by about 2 points on GPQA-Diamond.

An academic scaling study (Apple)

- ▶ 457 models, trained from scratch.
- ▶ At equal compute, early fusion is **on par** with late fusion, and slightly better at small budgets.
- ▶ Both follow almost the same scaling exponent as text-only models:

	$L \propto C^x$
Early fusion	-0.0492
Late fusion	-0.0494
GPT-3 (text)	-0.048

Late fusion's real advantage is *reuse* of pre-trained parts. With enough native training, that advantage shrinks.

Kimi Team, [arXiv:2602.02276](https://arxiv.org/abs/2602.02276), Tables 1 and 2. Shukor et al., "Scaling Laws for Native Multimodal Models," ICCV 2025, [arXiv:2504.07951](https://arxiv.org/abs/2504.07951).

- ▶ If one backbone handles every modality, should some weights be reserved for each?
- ▶ **Mixture-of-Transformers (Meta)**. Give each modality its own feed-forward, attention and normalisation weights, but keep *one global self-attention* over the whole sequence. Routing is by modality, with no learned router. It matches a dense Chameleon-style model at **55.8%** of the FLOPs.
- ▶ **Learned MoE (Apple)**. Let the router decide. Routing that ignores modality labels beats routing that uses them. Experts still specialise by modality on their own, mostly in the first and last layers.
- ▶ These agree more than they differ:

Specialise the capacity. Keep the attention global. It is the same division of labour as MoE in a text model: experts differ, mixing is shared.

Liang et al., “Mixture-of-Transformers,” [arXiv:2411.04996](#). Shukor et al., [arXiv:2504.07951](#).

- ▶ An image can cost over a thousand tokens. A video costs that per frame. With long context expensive, **visual-token compression** became a design axis of its own.
 - **Budgets:** Gemma 4 lets the caller choose 70 to 1,120 tokens per image.
 - **Temporal pooling:** Kimi K2.5 averages groups of four video frames at the patch level, and fits about $4\times$ longer video.
 - **Optical compression:** DeepSeek-OCR reads a page of text from an image with about 97% precision at under $10\times$ fewer tokens than the text itself. At $20\times$ it falls to about 60%.
- ▶ **Does the encoder have to exist?** Gemma 4's 12B model has none. Raw 48×48 pixel patches go through a single linear projection into the LLM. Audio chunks are projected the same way.

No published ablation yet compares encoder-free and encoder-based models at matched size. The other Gemma 4 sizes keep a ViT.

Gemma Team, [arXiv:2607.02770](https://arxiv.org/abs/2607.02770). Kimi Team, [arXiv:2602.02276](https://arxiv.org/abs/2602.02276). Wei et al., "DeepSeek-OCR: Contexts Optical Compression," [arXiv:2510.18234](https://arxiv.org/abs/2510.18234).

How images are generated	Example	GenEval $\uparrow$	MMMU $\uparrow$
Next-token prediction over image tokens	Emu3.5 (34B)	0.86	n/a
Autoregressive text, flow-matching images	BAGEL (14B)	0.82	55.3
Masked discrete diffusion for everything	LLaDA2.0-Uni (16B)	0.89	50.1
Multimodal LLM with a diffusion head	InternVL-U (4B)	0.85	54.7

- **No design has won.** Four very different mechanisms land within a few points of each other. Sizes and settings differ, so read the numbers as indicative.
- **Do the two tasks help each other?** A 2026 analysis says yes at the level of representations, and no if both are forced through identical computation. Share the knowledge. Decouple the pathway.

Scores as reported in: ByteDance Seed, "BAGEL," [arXiv:2505.14683](#). [arXiv:2603.09877](#) (InternVL-U). [arXiv:2604.20796](#) (LLaDA2.0-Uni). [arXiv:2601.02204](#) (NextFlow, for Emu3.5). [arXiv:2609.01607](#).

- ▶ The flagship of 2026 is **one natively multimodal model**, built on the same hybrid and MoE backbones as text models.
- ▶ **Fuse early in training.** A vendor ablation and a 457-model scaling study point the same way.
- ▶ **Specialise capacity, share attention.** Modality-specific experts with global mixing.
- ▶ **Visual tokens are a budget** to be compressed and chosen, like context length.
- ▶ **Unified generation is unsettled.** Autoregression, flow matching and discrete diffusion all remain in play.

Closed labs disclose almost no architectural detail. What we can teach rigorously comes from open-weight reports.

Pressure	The 2026 answer	Still open
KV cache grows with context	hybrids: recurrent mixers plus a few attention layers	recall and state tracking in a fixed state
Every token pays for every weight	MoE with 3 to 5% of weights active	routing score, shared experts, serving at small batch
Attention is quadratic	FlashAttention-3 and 4, learned sparse attention	the cost of the selector itself
Context beyond training length	YaRN, NoPE layers, explicit sinks, staged training	uniform quality across 1M tokens
Finite compute and data	over-train small models, curate data, shift compute to RL and inference	laws for post-training and for verifiers
Inputs beyond text	one natively multimodal model, fused early	unified generation, encoder-free inputs

The common thread: **do the expensive, exact operation rarely, and the cheap, approximate one everywhere else.**

- ▶ Hybrids and sparse attention both claim the long-context frontier. Under matched training and serving budgets, **which wins on multi-hop reasoning**, and why?
- ▶ Can a recurrent state be made to **grow on demand**, so capacity follows the input instead of being fixed in advance?
- ▶ What is the right **benchmark** for a million-token context, now that needle tests are saturated and vendor tables disagree?
- ▶ Sparse models lose on reading comprehension at matched loss. **What does loss fail to measure?**
- ▶ Is there a scaling law that jointly allocates compute between **pre-training, reinforcement learning and inference**?

Tomorrow

Day 2 picks up the last question: reasoning, search and verification at test time.